

APPLIED DATA STRUCTURE & ALGORITHMS LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RAUSHAN RAJ

SUBMMITED BY:

Shivang
(BCS2024352)
SECTION: D

Assignment 1: Reading Array Values

AIM: Read values from an array by index and by a loop.

TASKS:

TASK1: Create the array int[] marks = {45, 67, 80, 52, 90}. Print First: followed by the first value and Last: followed by the last value, on two lines, using indexes

IMPLEMENT:

```
public class Main {
    public static void main(String[] args) {

        int[] marks = {45, 67, 80, 52, 90};

        System.out.println("First: " + marks[0]);
        System.out.println("Last: " + marks[4]);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment1> javac Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment1> java Task1.java
45
90
```

TASK1:

Create the array int[] marks = {45, 67, 80, 52, 90}. Print every value on its own line, using a for loop.

```
public class Main {
    public static void main(String[] args) {

        int[] marks = {45, 67, 80, 52, 90};

        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }
    }
}
```

OUTPUT:
```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment1> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment1> java Task2.java
45
67
80
52
90
```

TASK3:

Create the array int[] marks = {45, 67, 80, 52, 90}. Add all the values with a loop and print the total as Sum: followed by the total.

```
public class Main {
    public static void main(String[] args) {
```

```
        int[] marks = {45, 67, 80, 52, 90};

        int sum = 0;

        for (int i = 0; i < marks.length; i++) {
            sum = sum + marks[i];
        }

        System.out.println("Sum: " + sum);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment1> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment1> java Task3.java
Sum: 334
```

Assignment 2: Largest, Smallest and Count.

AIM:: Find the largest and smallest value in an array and count values that pass a test.

TASKS::

TASK1:

Create the array int[] marks = {45, 67, 80, 52, 90}. Find the largest value with a loop and print it as Largest: followed by the value.

```
public class Main {
    public static void main(String[] args) {

        int[] marks = {45, 67, 80, 52, 90};

        int largest = marks[0];

        for (int i = 1; i < marks.length; i++) {

            if (marks[i] > largest) {
                largest = marks[i];
            }
        }

        System.out.println("Largest: " + largest);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> cd Assignment2

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> javac Task1.java

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> java Task1.java
```

Largest: 90

TASK2:

Create the array int[] marks = {45, 67, 80, 52, 90}. Find the smallest value with a loop and print it as Smallest: followed by the value

```
public class Main {
    public static void main(String[] args) {

        int[] marks = {45, 67, 80, 52, 90};

        int smallest = marks[0];

        for (int i = 1; i < marks.length; i++) {

            if (marks[i] < smallest) {
                smallest = marks[i];
            }
        }

        System.out.println("Smallest: " + smallest);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> ^C
```

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> java Task2.java
Smallest: 45
```

TASK3:

Create the array int[] marks = {45, 67, 80, 52, 90}. Count the values greater than 60 and print the count as Count: followed by the number.

```
public class Main {
    public static void main(String[] args) {

        int[] marks = {45, 67, 80, 52, 90};

        int count = 0;

        for (int i = 0; i < marks.length; i++) {

            if (marks[i] > 60) {
                count++;
            }
        }

        System.out.println("Count: " + count);
    }
```

```
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> javac Task2.java

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> javac Task3.java

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment2> java Task3.java

Count: 3
```

Assignment 3: Bubble Sort

AIM: Sort an array by comparing neighbouring values and swapping them when they are out of order

TASKS:

TASK1:

Run one pass of bubble sort on {5, 1, 4, 2, 8}. In the pass, compare each value with the next and swap when the left one is larger. Print each value of the array on its own line.

```
public class Main {
    public static void main(String[] args) {

        int[] arr = {5, 1, 4, 2, 8};

        for (int i = 0; i < arr.length - 1; i++) {
```

```
            if (arr[i] > arr[i + 1]) {

                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }

        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment3> javac Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment3> java Task1.java
1
4
2
5
8
```

TASK2:

Sort the array {5, 1, 4, 2, 8} with bubble sort, using one outer loop for the passes and one inner loop for the comparisons.

Print each value of the sorted array on its own line.

```
public class Main {
   public static void main(String[] args) {

      int[] arr = {5, 1, 4, 2, 8};

      for (int i = 0; i < arr.length - 1; i++) {

         for (int j = 0; j < arr.length - 1 - i; j++) {

            if (arr[j] > arr[j + 1]) {

               int temp = arr[j];
               arr[j] = arr[j + 1];
               arr[j + 1] = temp;
            }
         }
      }

      for (int i = 0; i < arr.length; i++) {
         System.out.println(arr[i]);
      }
   }
```

```
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment3> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment3> java Task2.java
1
2
4
5
8
```

TASK3:

Sort the array {9, 7, 3} with bubble sort and print each value of the sorted array on its own line.

```
public class Main {
   public static void main(String[] args) {

      int[] arr = {9, 7, 3};

      for (int i = 0; i < arr.length - 1; i++) {

         for (int j = 0; j < arr.length - 1 - i; j++) {

            if (arr[j] > arr[j + 1]) {
```

```
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    for (int i = 0; i < arr.length; i++) {
        System.out.println(arr[i]);
    }
  }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment3> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment3> java Task3.java
3
7
9
```

Assignment 4: Selection Sort.

AIM: **Sort an array by finding the smallest value in the unsorted part and moving it to the front.**

TASKS:

TASK1:

Find the index of the smallest value in the array {64, 25, 12, 22, 11} with a loop, and print that index.

```
public class Main {
   public static void main(String[] args) {

      int[] arr = {64, 25, 12, 22, 11};

      int minIndex = 0;

      for (int i = 1; i < arr.length; i++) {

         if (arr[i] < arr[minIndex]) {
            minIndex = i;
         }
      }

      System.out.println("Index: " + minIndex);
   }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> cd Assignment4

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment4> javac Task1.java
```

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment4> java Task1.java
Index: 4
```

TASK2:

. Sort the array {64, 25, 12, 22, 11} with selection sort. For each position, swap in the smallest value of the rest. Print each value of the sorted array on its own line.

```
public class Main {
    public static void main(String[] args) {

        int[] arr = {64, 25, 12, 22, 11};

        for (int i = 0; i < arr.length - 1; i++) {

            int minIndex = i;

            for (int j = i + 1; j < arr.length; j++) {

                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            int temp = arr[i];
            arr[i] = arr[minIndex];
```

```
            arr[minIndex] = temp;
        }

        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment4> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment4> java Task2.java
11
12
22
25
64
```

TASK3:

Do only the first step of selection sort on {64, 25, 12, 22, 11}: find the smallest value, swap it with the first value, and print each value of the array on its own line.

```
public class Main {
    public static void main(String[] args) {
```

```
        int[] arr = {64, 25, 12, 22, 11};

        int minIndex = 0;

        for (int i = 1; i < arr.length; i++) {

            if (arr[i] < arr[minIndex]) {
                minIndex = i;
            }
        }

        int temp = arr[0];
        arr[0] = arr[minIndex];
        arr[minIndex] = temp;

        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment4> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment4> java Task3.java
```

11

25

12

22

64

Assignment 5: Linear Search

AIM: : Search an array one value at a time and report the index where a value is found.

TASKS::

TASK1:

Write the method int linearSearch(int[] a, int key) that returns the index of key, or -1 when key is not in the array. Call it with {45, 67, 80, 52, 90} and key 80, and print the result.

```
public class Main {

    static int linearSearch(int[] a, int key) {

        for (int i = 0; i < a.length; i++) {

            if (a[i] == key) {
                return i;
            }
```

```
        }

        return -1;
    }

    public static void main(String[] args) {

        int[] arr = {45, 67, 80, 52, 90};

        int result = linearSearch(arr, 80);

        System.out.println("Index: " + result);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> cd Assignment5
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment5> javac Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment5> java Task1.java
Index: 2
```

TASK2:

Write the method int linearSearch(int[] a, int key) that returns the index of key, or -1 when key is not in the

array. Call it with {45, 67, 80, 52, 90} and key 100, and print the result.

```
public class Main {

   static int linearSearch(int[] a, int key) {

      for (int i = 0; i < a.length; i++) {

         if (a[i] == key) {
            return i;
         }
      }

      return -1;
   }

   public static void main(String[] args) {

      int[] arr = {45, 67, 80, 52, 90};

      int result = linearSearch(arr, 100);

      System.out.println("Index: " + result);
   }
}
```

OUTPUT::

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment5> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment5> java Task2.java
Index: -1
```

TASK3::

Count how many times the value 52 appears in the array {52, 10, 52, 7, 52}. Print the count as Count: followed by the number.

```
public class Main {
    public static void main(String[] args) {

        int[] arr = {52, 10, 52, 7, 52};

        int count = 0;

        for (int i = 0; i < arr.length; i++) {

            if (arr[i] == 52) {
                count++;
            }
        }
```

```
        System.out.println("Count: " + count);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment5> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment5> java Task3.java
Count: 3
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment5>
```

Assignment 6: Binary Search on a Sorted Array.

AIM: Search a sorted array by checking the middle value and throwing away half of the array each time.

TASKS::

TASK1:

Write the method int binarySearch(int[] a, int key) for a sorted array, returning the index of key or -1. Call it on {10, 20, 30, 40, 50} with key 40 and print the result.

```
public class Main {
```

```
static int binarySearch(int[] a, int key) {

    int low = 0;
    int high = a.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (a[mid] == key) {
            return mid;
        }
        else if (a[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    return -1;
}

public static void main(String[] args) {
```

```
        int[] arr = {10, 20, 30, 40, 50};

        int result = binarySearch(arr, 40);

        System.out.println("Index: " + result);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> cd Assignment6
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment6> javac Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment6> java Task1.java
Index: 3
```

TASK2:

Write the method int binarySearch(int[] a, int key) for a sorted array, returning the index of key or -1. Call it on {10, 20, 30, 40, 50} with key 35 and print the result.

```
public class Main {

    static int binarySearch(int[] a, int key) {

        int low = 0;
```

```
    int high = a.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (a[mid] == key) {
            return mid;
        }
        else if (a[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    return -1;
}

public static void main(String[] args) {

    int[] arr = {10, 20, 30, 40, 50};

    int result = binarySearch(arr, 35);
```

```
        System.out.println("Index: " + result);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment6> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment6> java Task2.java
Index: -1
```

TASK3:

Write the method int binarySearch(int[] a, int key) for a sorted array, returning the index of key or -1. Call it on {10, 20, 30, 40, 50} with key 10 and print the result.

```
public class Main {

    static int binarySearch(int[] a, int key) {

        int low = 0;
        int high = a.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;
```

```
            if (a[mid] == key) {
                return mid;
            }
            else if (a[mid] < key) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }

        return -1;
    }

    public static void main(String[] args) {

        int[] arr = {10, 20, 30, 40, 50};

        int result = binarySearch(arr, 10);

        System.out.println("Index: " + result);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment6> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment6> java Task3.java
Index: 0
```

Assignment 7: Practice Set

AIM: Solve four short problems from arrays, searching, sorting and loops

TASKS:

TASK1:

Search for the value 52 in the array {45, 52, 80} with a loop. If 52 is there, print Found at index followed by a space and the index. If it is not there, print Not found.

```
public class Main {
    public static void main(String[] args) {

        int[] arr = {45, 52, 80};

        int key = 52;
        boolean found = false;

        for (int i = 0; i < arr.length; i++) {
```

```
            if (arr[i] == key) {
                System.out.println("Found at index " + i);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Not found");
        }
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment7> javac Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment7> java Task1.java
Found at index 1
```

TASK2:

Sort the array {3, 1, 2} with bubble sort and print each value of the sorted array on its own line.

```
public class Main {
    public static void main(String[] args) {
```

```
        int[] arr = {3, 1, 2};

        for (int i = 0; i < arr.length - 1; i++) {

            for (int j = 0; j < arr.length - 1 - i; j++) {

                if (arr[j] > arr[j + 1]) {

                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment7> javac Task2.java
```

```
PS C:\Users\acer\Desktop\Java-DSA-
Assignments\Assignment7> java Task2.java
1
2
3
```

TASK3::

Use a for loop to multiply all the values of {2, 3, 4} and print the product as Product: followed by the number.

```
public class Main {
    public static void main(String[] args) {

        int[] arr = {2, 3, 4};

        int product = 1;

        for (int i = 0; i < arr.length; i++) {

            product = product * arr[i];
        }

        System.out.println("Product: " + product);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment7> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment7> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment7> java Task3.java
Product: 24
```

Assignment 8: Classes and Objects

AIM: Write a small class with fields and a method, and create objects from it.

TASKS:

TASK1:

Create class Student with int roll, String name and method show() that prints Roll: and the roll, then Name: and the name. In main, create a Student with roll 5 and name Rohan and call show().

```
class Student {
    int roll;
    String name;

    void show() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
```

```
    }
}


public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();


        s1.roll = 5;
        s1.name = "Rohan";


        s1.show();
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment8\Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment8.Task1
Roll: 101
Name: Rama
```

TASK2:

Create class Student with int roll, String name and method show() that prints Roll: and the roll, then Name: and the name. Create two Students: roll 5 named Rohan, and roll 8 named Meera. Call show() on each.

```
class Student {

    int roll;
    String name;

    void show() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();

        s1.roll = 5;
        s1.name = "Rohan";

        Student s2 = new Student();

        s2.roll = 8;
        s2.name = "Meera";

        s1.show();
```

```
        s2.show();
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment8\Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment8.Task2
Roll: 5
Name: Rohan
Roll: 8
Name: Meera
```

TASK3:

Create class Book with int pages and method isLong() that returns true when pages is more than 300. In main, create a Book with 450 pages and one with 120 pages, and print isLong() for each, one per line.

```
class Book {

    int pages;

    boolean isLong() {
        return pages > 300;
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Book b1 = new Book();
        b1.pages = 450;

        Book b2 = new Book();
        b2.pages = 120;

        System.out.println(b1.isLong());
        System.out.println(b2.isLong());
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment8\Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment8.Task3
true
false
```

Assignment 9: Constructors.

AIM:

Set field values when an object is created, using constructors.

TASKS:

TASK1:

Create class Box with int field width and a constructor with no parameters that sets width to 10. In main, create a Box and print Width: followed by the value.

```
class Box {

    int width;

    Box() {
        width = 10;
    }
}

public class Main {

    public static void main(String[] args) {

        Box b1 = new Box();

        System.out.println("Width: " + b1.width);
}
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment9\Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment9.Task1
Width: 10
```

TASK2:

Create class Box with int field width and a constructor that takes an int and sets width using this.width. In main, create a Box with 25 and print Width: followed by the value.

```
class Box {

    int width;

    Box(int width) {
        this.width = width;
    }
}

public class Main {

    public static void main(String[] args) {

        Box b1 = new Box(25);

        System.out.println("Width: " + b1.width);
```

```
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment9\Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment9.Task2
Width: 25
```

TASK3: Create class Counter with int field count, set to 0 by its constructor, and method increment() that adds 1 to count. In main, create one Counter, call increment() three times, and print Count: followed by the value..

```
class Counter {

    int count;

    Counter() {
        count = 0;
    }

    void increment() {
        count++;
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        Counter c1 = new Counter();

        c1.increment();
        c1.increment();
        c1.increment();

        System.out.println("Count: " + c1.count);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment9\Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment9.Task3
Count: 3
```

Assignment 10: Parent and Child Classes.

AIM:: Make one class extend another, so that the child class uses the parent's fields and methods.

TASKS::

TASK1: Create class Animal with method sound() that prints 'Some sound'. Create class Dog that extends Animal with no other code. In main, create a Dog and call sound() on it.

```
class Animal {

    void sound() {
        System.out.println("Some sound");
    }
}

class Dog extends Animal {

}

public class Main {

    public static void main(String[] args) {

        Dog d1 = new Dog();

        d1.sound();
    }
```

```
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment8\Task1.java

PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment10\Task1.java

PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment10.Task1

Some sound
```

TASK2: Create class Person with field name, set by a constructor. Create class Student extending Person, whose constructor takes a name and calls super(name). In main, create a Student named Neha and print Name: followed by the name.

```
class Person {

    String name;

    Person(String name) {
        this.name = name;
    }
}

class Student extends Person {
```

```
    Student(String name) {
        super(name);
    }
}

public class Main {

    public static void main(String[] args) {

        Student s1 = new Student("Neha");

        System.out.println("Name: " + s1.name);
    }
}
```

OUTPUT:: PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment10\Task2.java

PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment10.Task2

Name: Neha

TASK3:: Create class Person with field name and method greet() that prints Hello, followed by the name. Create class Teacher extending Person with no other code. In main, create a Teacher, set its name to Kiran and call greet().

```
class Person {
    String name;
    void greet() {
        System.out.println("Hello, " + name);
    }
}
class Teacher extends Person {
}
public class Main {
    public static void main(String[] args) {
        Teacher t1 = new Teacher();
        t1.name = "Kiran";
        t1.greet();
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment10\Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment10.Task3
Hello, Kiran
```

Assignment 11: Method Overriding.

AIM: Change a method in a child class so that it gives a different answer from the parent version.

TASKS::

TASK1: Create class Shape with int area() returning 0, and class Square extending Shape with int field side and area() returning side times side. In main, create a Square, set side to 4, and print Area: followed by the value.

```
class Shape {

    int area() {
        return 0;
    }
}

class Square extends Shape {

    int side;

    @Override
    int area() {
        return side * side;
    }
}

public class Main {
```

```
    public static void main(String[] args) {

        Square s1 = new Square();

        s1.side = 4;

        System.out.println("Area: " + s1.area());
    }
}
```

OUTPUT::

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment11\Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment11.Task1
Area: 16
```

TASK2: Create class Shape with int area() returning 0, and class Rectangle extending Shape with int length and breadth, area() returning length times breadth. In main, create Rectangle with length 4 and breadth 5, and print Area: followed by the value

```
class Shape {

    int area() {
        return 0;
    }
}
```

```
class Rectangle extends Shape {

    int length;
    int breadth;

    @Override
    int area() {
        return length * breadth;
    }
}

public class Main {

    public static void main(String[] args) {

        Rectangle r1 = new Rectangle();

        r1.length = 4;
        r1.breadth = 5;

        System.out.println("Area: " + r1.area());
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment11\Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment11.Task2
Area: 20
```

TASK3: Create class Shape with int area() returning 0, class Square extending Shape with int side, area() returning side times side. In main, create Square with side 3, store it in a Shape variable and print Area: followed by the value.

```
class Shape {

    int area() {
        return 0;
    }
}

class Square extends Shape {

    int side;

    @Override
    int area() {
        return side * side;
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        Shape s1 = new Square();

        Square sq = (Square) s1;
        sq.side = 3;

        System.out.println("Area: " + s1.area());
    }
}
```

OUTPUT::

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> javac Assignment11\Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments> ava Assignment11.Task3
PS C:\Users\acer\Desktop\Java-DSA-Assignments> java Assignment11.Task3
Area: 9
```

Assignment 12: Recursive Factorial, Sum and Power.

AIM: : Write a method that calls itself on a smaller input until a base case is reached.

TASKS::

TASK1: Write the method int factorial(int n) using recursion. The factorial of 0 is 1, and n times factorial(n - 1) is the other case. Print factorial(5).

```
public class Main {

   static int factorial(int n) {

      if (n == 0) {
         return 1;  // Base case
      }

      return n * factorial(n - 1);
   }

   public static void main(String[] args) {

      System.out.println("Factorial of 5: " + factorial(5));
   }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> cd Assignment12

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment12> javac Task1.java
```

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment12> java Task1.java
Factorial of 5: 120
```

TASK2: Write the method int sum(int n) using recursion that adds the numbers from 1 to n. Print sum(10).

```
public class Main {

   static int sum(int n) {

      if (n == 0) {
         return 0;  // Base case
      }

      return n + sum(n - 1);
   }

   public static void main(String[] args) {

      System.out.println("Sum of 10: " + sum(10));
   }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment12> javac Task2.java
Sum of 10: 55
```

TASK3: Write the method int power(int base, int exp) using recursion. When exp is 0 the answer is 1; otherwise the answer is base times power(base, exp - 1). Print power(2, 5).

```
public class Main {

   static int power(int base, int exp) {

      if (exp == 0) {
         return 1;  // Base case
      }

      return base * power(base, exp - 1);
   }

   public static void main(String[] args) {

      System.out.println("Power: " + power(2, 5));
   }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment12> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment12> java Task3.java
Power: 32
```

Assignment 13: Recursion on Arrays

AIM: Walk through an array with an index that goes one step further in each call.

TASKS::

TASK1: Write the method int sum(int[] a, int i) that returns the sum of the values from index i to the end. Call it on {3, 6, 9, 12, 15} with i = 0 and print the result.

```
public class Main {

    static int sum(int[] a, int i) {

        if (i == a.length) {
            return 0;  // Base case
        }

        return a[i] + sum(a, i + 1);
    }

    public static void main(String[] args) {

        int[] a = {3, 6, 9, 12, 15};
```

```
        System.out.println("Sum: " + sum(a, 0));
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment13> javac Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment13> java Task1.java
Sum: 45
```

TASK2: Write the method int max(int[] a, int i) that returns the largest value from index i to the end. Call it on {3, 6, 9, 12, 15} with i = 0 and print the result.

```
public class Main {

    static int max(int[] a, int i) {

        if (i == a.length - 1) {
            return a[i];  // Base case
        }

        int result = max(a, i + 1);

        return Math.max(a[i], result);
```

```
    }

    public static void main(String[] args) {

        int[] a = {3, 6, 9, 12, 15};

        System.out.println("Maximum: " + max(a, 0));
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment13> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment13> java Task2.java
Maximum: 15
```

TASK3: Write the method void printReverse(int[] a, int i) that prints the values from index i down to 0, one per line, using recursion. Call it on {3, 6, 9, 12, 15} with i set to the last index.

```
public class Main {

    static void printReverse(int[] a, int i) {

        if (i < 0) {
            return;  // Base case
        }
```

```
        System.out.println(a[i]);

        printReverse(a, i - 1);
    }

    public static void main(String[] args) {

        int[] a = {3, 6, 9, 12, 15};

        printReverse(a, a.length - 1);
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment13> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment13> java Task3.java
15
12
9
6
3
```

Assignment 14: Sorted Check, Search and Fibonacci.

AIM:: Use recursion to check an array, find a value by index, and compute Fibonacci numbers.

TASKS::

TASK1: . Write the method boolean isSorted(int[] a, int i) that returns true when the values from index i to the end are in ascending order. Call it on {2, 4, 6, 8, 10} with i = 0 and print the result.

```
public class Main {

    static boolean isSorted(int[] a, int i) {

        if (i == a.length - 1) {
            return true;  // Base case
        }

        if (a[i] > a[i + 1]) {
            return false;
        }

        return isSorted(a, i + 1);
```

```
    }

    public static void main(String[] args) {

        int[] a = {2, 4, 6, 8, 10};

        System.out.println("Is Sorted: " + isSorted(a, 0));
    }
}
```

OUTPUT::

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> cd Assignment14
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment14> javac Task1.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment14> java Task1.java
Is Sorted: true
```

TASK2: Write the method int findIndex(int[] a, int key, int i) that returns the index of key from index i onwards, or -1 if absent. Call it on {45, 67, 80, 52, 90}, key 80, i = 0, and print it.

```
public class Main {

    static int findIndex(int[] a, int key, int i) {
```

```
        if (i == a.length) {
            return -1;  // Base case
        }

        if (a[i] == key) {
            return i;
        }

        return findIndex(a, key, i + 1);
    }

    public static void main(String[] args) {

        int[] a = {45, 67, 80, 52, 90};

        System.out.println("Index: " + findIndex(a, 80, 0));
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment14> javac Task2.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment14> java Task2.java
Index: 2
```

TASK3: Write the method int fib(int n) using recursion, where fib(0) is 0, fib(1) is 1, and otherwise fib(n) is fib(n - 1) plus fib(n - 2). Print fib(7).

```
public class Main {

  static int fib(int n) {

    if (n == 0) {
      return 0;  // Base case
    }

    if (n == 1) {
      return 1;  // Base case
    }

    return fib(n - 1) + fib(n - 2);
  }

  public static void main(String[] args) {

    System.out.println("Fibonacci: " + fib(7));
  }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment14> javac Task3.java
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment14> java Task3.java
Fibonacci: 13
```

Assignment 15: Subsets and Arrangements.

AIM: Build answers one choice at a time, print them, and undo each choice before the next one.

TASKS:

TASK1: Print all subsets of {1, 2, 3} using recursion. At each index, first skip the value, then include it. At the end, print the subset in braces, for example {1, 3}, and print {} for the empty subset.

```
import java.util.*;


public class Main {


   static void subsets(int[] arr, int index, ArrayList<Integer> current) {


       // Base case
       if (index == arr.length) {
          System.out.println(current);
          return;
```

```
        }

        // First: Skip the element
        subsets(arr, index + 1, current);

        // Second: Include the element
        current.add(arr[index]);
        subsets(arr, index + 1, current);

        // Backtrack: Remove the element
        current.remove(current.size() - 1);
    }

    public static void main(String[] args) {

        int[] arr = {1, 2, 3};

        subsets(arr, 0, new ArrayList<>());
    }
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments> cd Assignment15

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment15> javac Task1.java
```

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment15>
java Task1.java
[]
[3]
[2]
[2, 3]
[1]
[1, 3]
[1, 2]
[1, 2, 3]
```

TASK2: Print all arrangements of the letters a, b and c. Use a used[] array: pick a letter that is not used, mark it used, fill the next position, then unmark it. Print each arrangement on its own line.

```
public class Main {

   static void arrange(char[] letters, boolean[] used,
                       String current) {

      // Base case
      if (current.length() == letters.length) {
         System.out.println(current);
         return;
      }

      for (int i = 0; i < letters.length; i++) {

         if (!used[i]) {
```

```
            // Choose letter
            used[i] = true;

            // Recursive call
            arrange(letters, used, current + letters[i]);

            // Backtrack
            used[i] = false;
        }
    }
}

public static void main(String[] args) {

    char[] letters = {'a', 'b', 'c'};
    boolean[] used = new boolean[3];

    arrange(letters, used, "");
}
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment15> javac Task2.java

PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment15> java Task2.java

abc
```

```
acb
bac
bca
cab
cba
```

TASK3: Print all arrangements of the letters x and y, one per line, using a used[] array: pick a letter that is not used, mark it used, fill the next position, then unmark it.

```
public class Main {

    static void arrange(char[] letters, boolean[] used,
                        String current) {

        // Base case
        if (current.length() == letters.length) {
            System.out.println(current);
            return;
        }

        for (int i = 0; i < letters.length; i++) {

            if (!used[i]) {
```

```
            // Choose letter
            used[i] = true;

            // Recursive call
            arrange(letters, used, current + letters[i]);

            // Backtrack
            used[i] = false;
        }
    }
}

public static void main(String[] args) {

    char[] letters = {'x', 'y'};
    boolean[] used = new boolean[2];

    arrange(letters, used, "");
}
}
```

OUTPUT:

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment15>
javac Task3.java
```

```
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment15> java Task3.java
xy
yx
PS C:\Users\acer\Desktop\Java-DSA-Assignments\Assignment15>
```